Introduction

When an application consumes messages from a Kafka broker it consumes the messages in batches. It is important then to understand what happens to the messages in the batch under different failure and exception conditions. This article walks through each scenario of interest explaining what happens under the covers of the Kafka client library.

Batch Consume

A Kafka consumer will be assigned zero or more partitions from across the one or more topics that it is subscribed to. While the most common use case may be for each consumer to subscribe to a single topic, there is no limit to the number of topics it may subscribe. Whichever topic partitions the consumer is assigned, when it polls the broker for messages it will receive a batch of messages that may span multiple topics and multiple partitions.

In order to compare the behaviour of a batch consume across different scenarios, in each case the batch will contain three messages. The first three messages to be polled are **alpha**, **bravo**, and **charlie**. The fourth message, **delta**, is not consumed in the first batch.

These examples assume that the consumer is configured to auto commit offsets. This is the default, configured via the **enable.auto.commit** parameter. If this is disabled it is in the developer's hands to manage committing the offsets, and hence potentially diverge from the default behaviour.

Scenario 1: Happy Path

The first scenario is the happy path flow for a consumer that is assigned a single partition on a single topic.

The Kafka client consumer polls the broker and receives the batch of messages. It then iterates over each message, **alpha**, **bravo**, and **charlie**, passing it to the configured listener. The listener processes each message in turn. Upon completion the Kafka client writes to the internal Kafka consumer offsets topic, writing the offset of **charlie**, the last message in the batch. This marks the messages as successfully consumed, and the next poll starts from the next offset after **charlie**, pulling in **delta** and beyond.

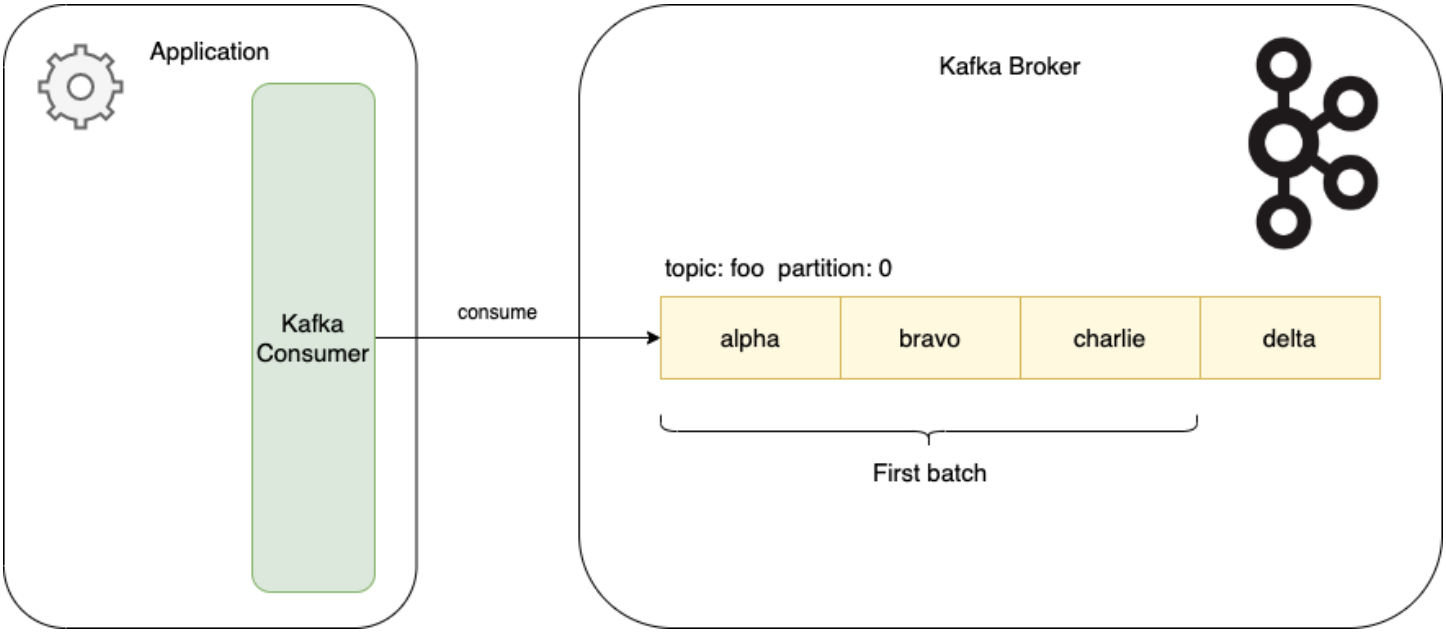

Figure 1: Message batch successfully consumed

The sequence diagram shows how the Kafka client library orchestrates the processing of the batch, from consuming it from the inbound topic to passing each message to the application consumer. Finally it writes to the consumer offsets topic to mark the batch as consumed (assuming the

configured **auto.commit.interval.ms** interval has elapsed since the last offsets write). As in this case the batch only contains messages from one partition, it writes the one record to the offsets topic with the offset of the last message processed from the batch, that of **charlie**.

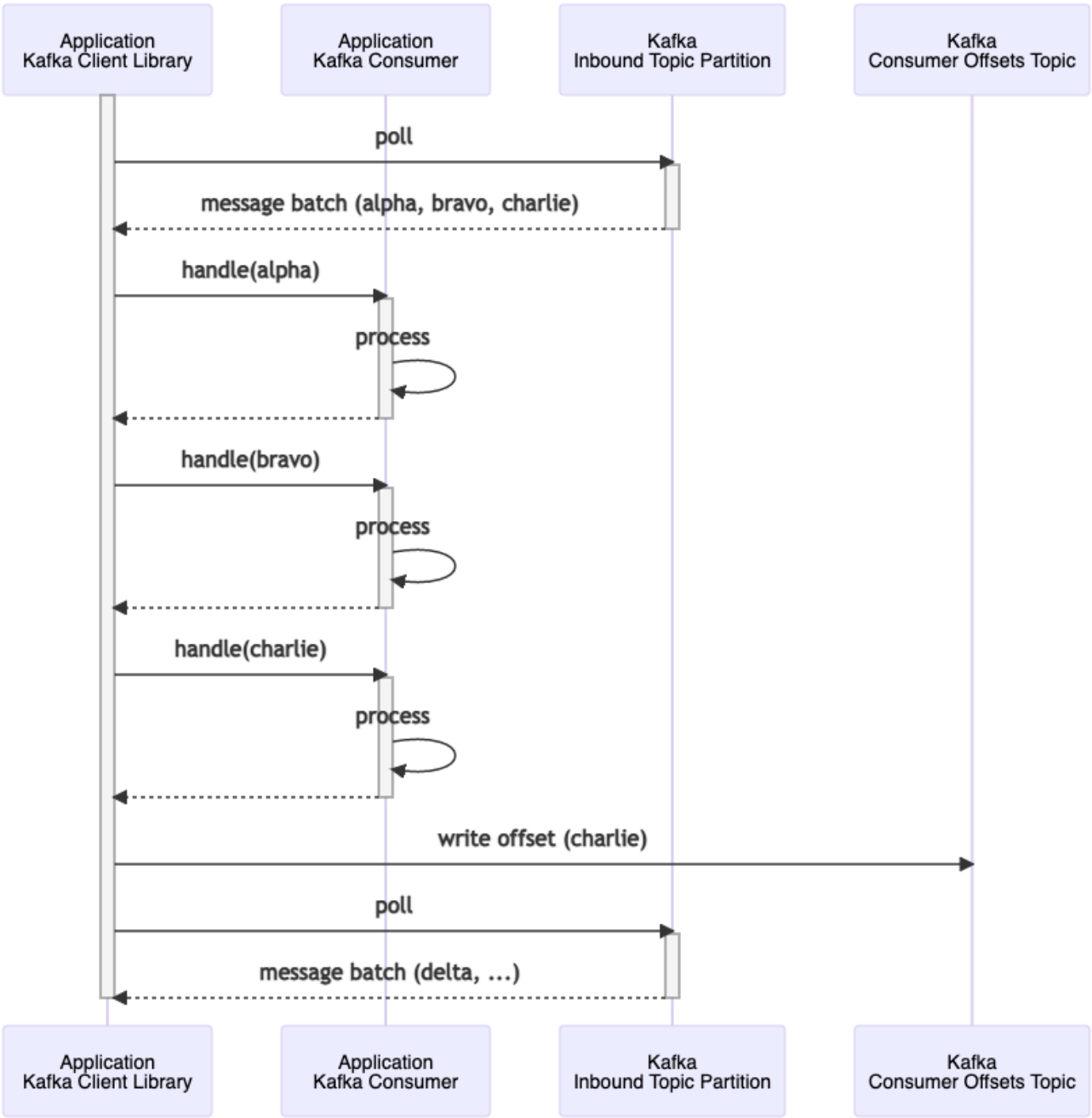

Figure 2: Message batch successfully consumed - sequence diagram

Scenario 2: Consumer Fails

In the second scenario the same batch of three messages is consumed. The first message, **alpha**, is successfully processed. During processing of the second message, **bravo**, the consumer dies. As the processing of the entire batch had not completed, the Kafka client library did not write the offset to the consumer offsets topic to mark the batch as consumed.

As the consumer has died and no longer heartbeats to the broker, a consumer group rebalance is triggered and the partition is assigned to another consumer instance (or indeed the same consumer instance if it restarts). Once the consumer group has stabilized and polling once again begins, the same batch of messages is redelivered. The

message **alpha** will be processed in full for a second time. The message **bravo**, which was partially processed before the consumer died, will be processed again, this time to completion. Finally, **charlie** will be processed for the first time. At this point the batch has completed, so the offset for **charlie** is written to the consumer offsets topic to mark the three messages as successfully consumed.

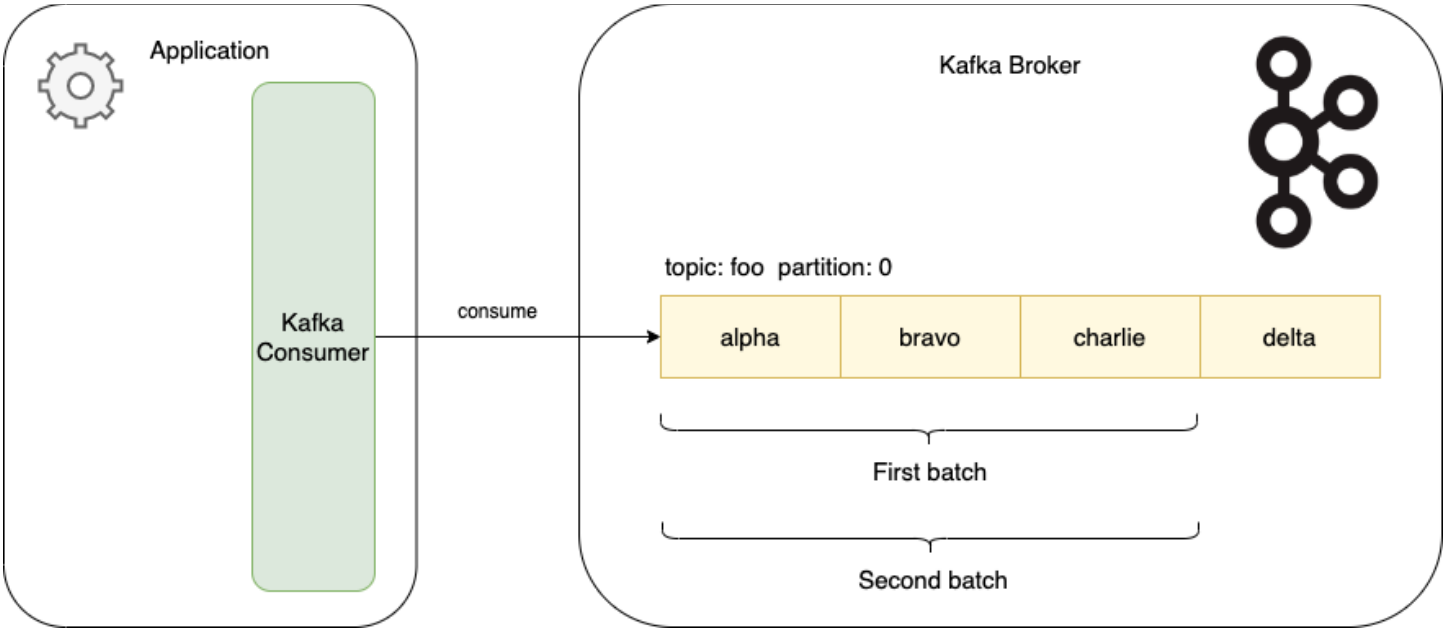

Figure 3: Consumer dies mid-batch

The sequence diagram shows the impact of the consumer dieing mid-processing. With auto commit enabled, the Kafka client library does not attempt to commit offsets until the end of the batch processing. This means that it will re-poll the same batch of messages when the application restarts.

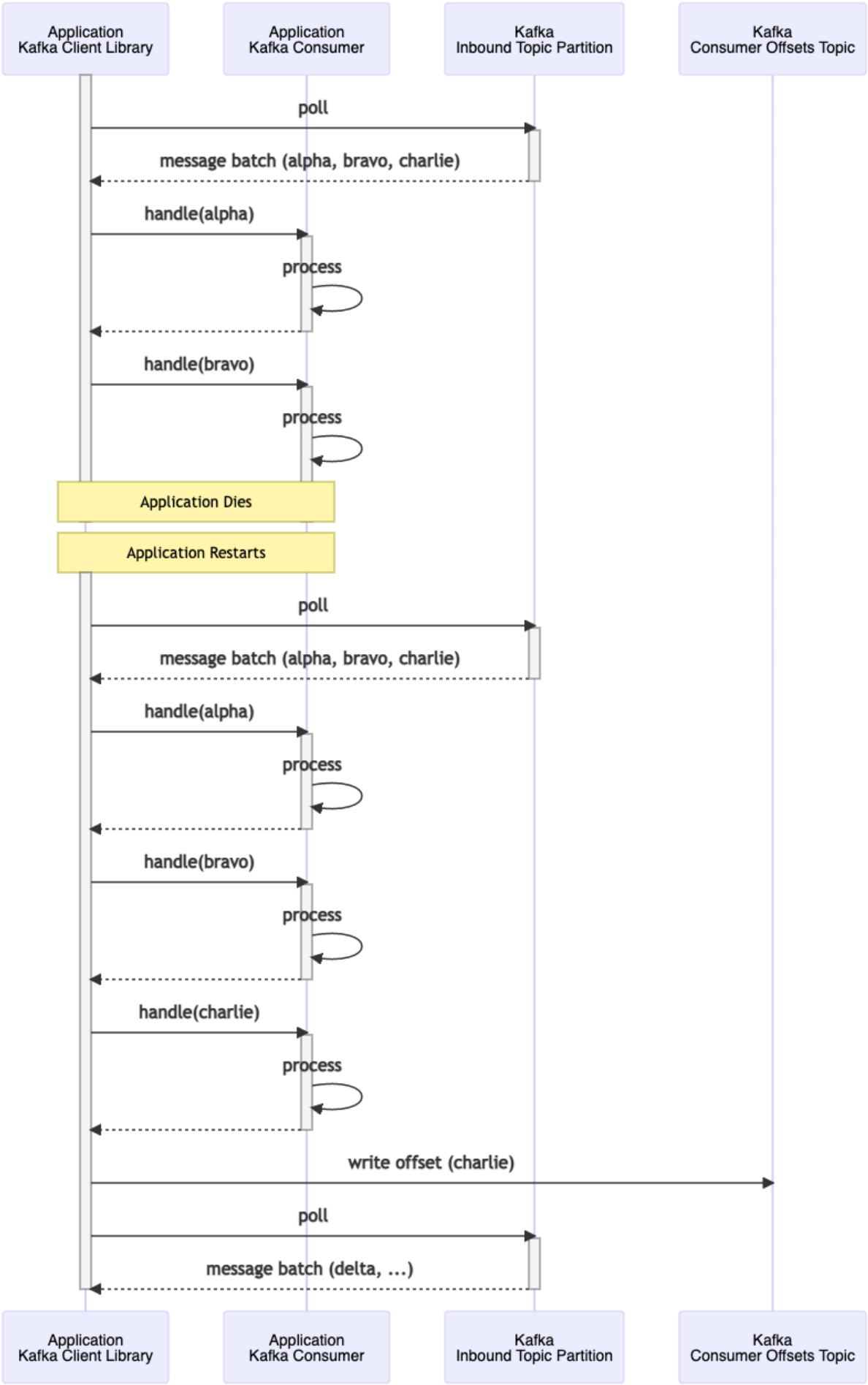
Application
Kafka Client Library
Application
Kafka Consumer
Kafka
Inbound Topic Partition
Kafka
Consumer Offsets Topic
poll
message batch (alpha, bravo, charlie)
handle(alpha)
process
handle(bravo)
process
Application Dies
Application Restarts
poll
message batch (alpha, bravo, charlie)
handle(alpha)
process
handle(bravo)
process
handle(charlie)
process
write offset (charlie)
poll
message batch (delta, ...)
Application
Kafka Client Library
Application
Kafka Consumer
Kafka
Inbound Topic Partition
Kafka
Consumer Offsets Topic

Figure 4: Consumer dies mid-batch - sequence diagram

Clearly then the possibility of duplicate messages being delivered must be considered. This refers to Kafka's guaranteed at-least-once message delivery. This is covered in the article [Kafka: Consume & Produce: At-Least-Once Delivery](#).

Scenario 3: Exception On Message Processing

The third scenario is where one message in the batch throws an exception during its processing. Depending on the type of exception and how it is handled will affect the processing of the message batch. If the exception is caught and is not allowed to percolate up to the Kafka client library, then the message is still considered as successfully processed, and processing moves on to the next message in the batch, just as with the happy path flow. A common scenario here is for a resulting message to be written to a dead message topic.

On the other hand, if the exception that is thrown is allowed to percolate up to the Kafka client library, the message is treated as having failed to have been successfully consumed. The intention here is to allow the message to be retried from the topic partition. An example might be the processing of the message results in a write to a database, but the database is temporarily unavailable, such that a retry might give it time to recover.

Similar to the consumer dies example, **alpha** is first processed successfully. The processing of **bravo** however results in an exception being thrown. At this point the Kafka client library catches this exception, and with auto commit enabled, writes the offset of **alpha** to the consumer offsets topic. The processing of the batch is now complete, and the Kafka client library polls the topic partition again from the latest offset. As the batch size is three messages , this time it contains **bravo**, **charlie**, and **delta**. The client passes **bravo** to the consumer which now successfully completes processing, followed by **charlie** and **delta** which likewise complete successfully. The batch processing is complete, and the offset for **delta** is written to the consumer offset topic.

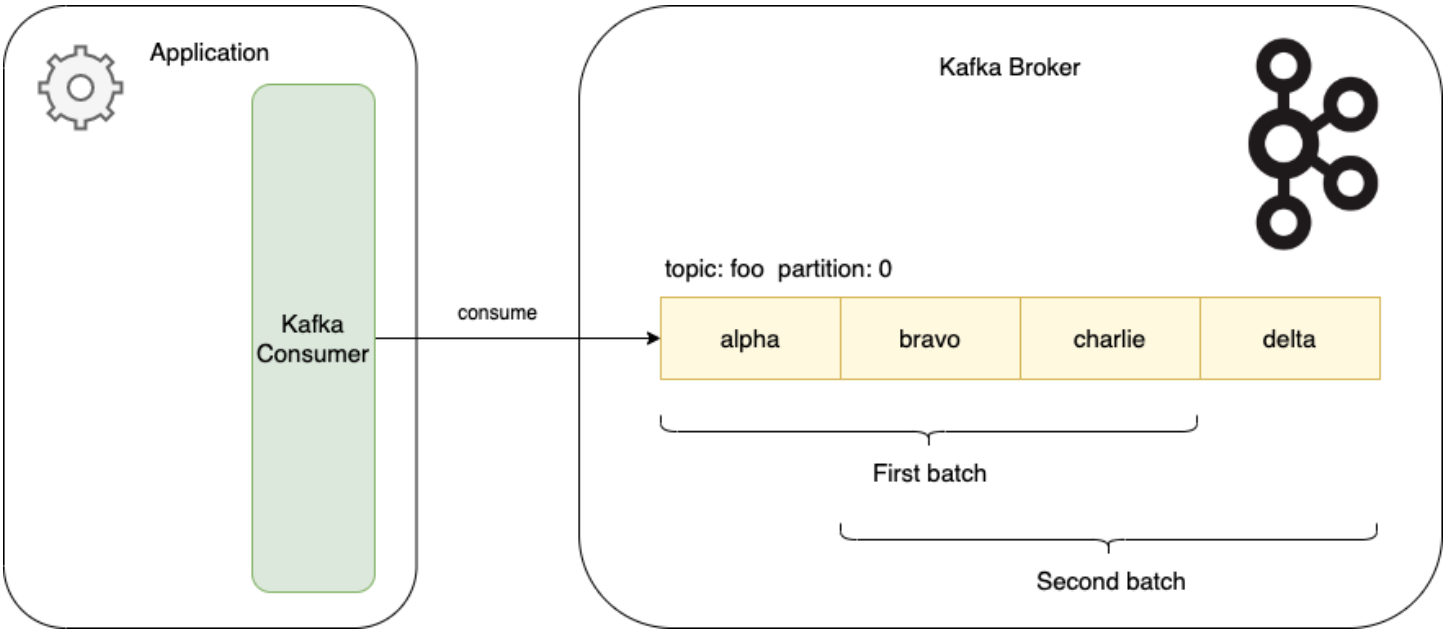

Figure 5: Batch message throws exception

The sequence diagram shows how the Kafka client library is able to catch and cleanly handle the exception, writing the offsets to the consumer offsets topic to mark the **alpha** message as successfully processed. It also shows how any processing that had taken place whilst processing the **bravo** message before it threw an exception will result in duplicate processing on the next poll.

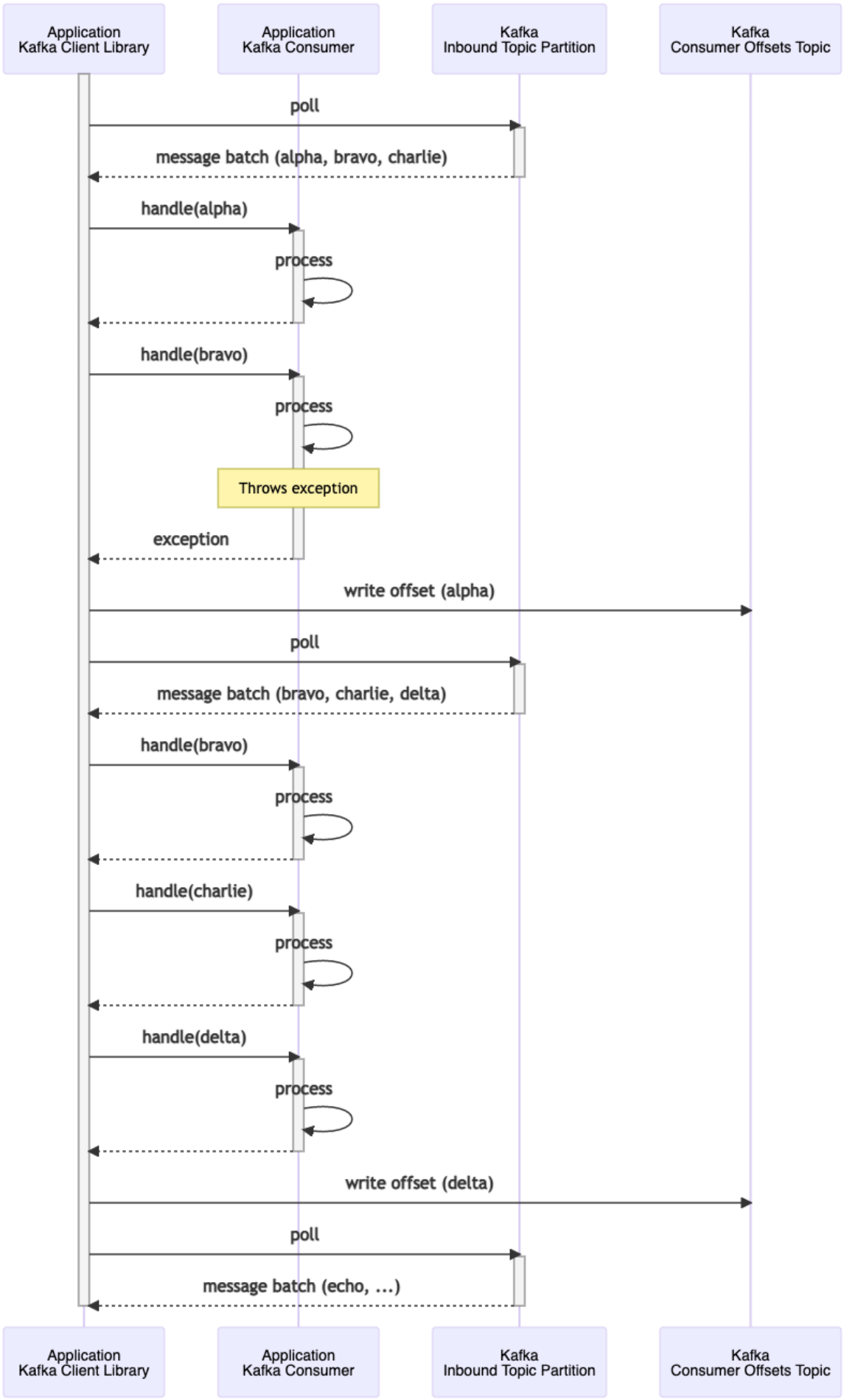
Application
Kafka Client Library
Application
Kafka Consumer
Kafka
Inbound Topic Partition
Kafka
Consumer Offsets Topic
poll
message batch (alpha, bravo, charlie)
handle(alpha)
process
handle(bravo)
process
Throws exception
exception
write offset (alpha)
poll
message batch (bravo, charlie, delta)
handle(bravo)
process
handle(charlie)
process
handle(delta)
process
write offset (delta)
poll
message batch (echo, ...)
Application
Kafka Client Library
Application
Kafka Consumer
Kafka
Inbound Topic Partition
Kafka
Consumer Offsets Topic

Figure 6: Batch message throws exception - sequence diagram

Batch Consume Spanning Topics & Partitions

As earlier noted, a consumer may be subscribed to multiple partitions in a topic, and to multiple topics. In the diagram below, the consumer is polling from partitions **0** and **1** of topic **foo** and partition **1** of topic **bar**.

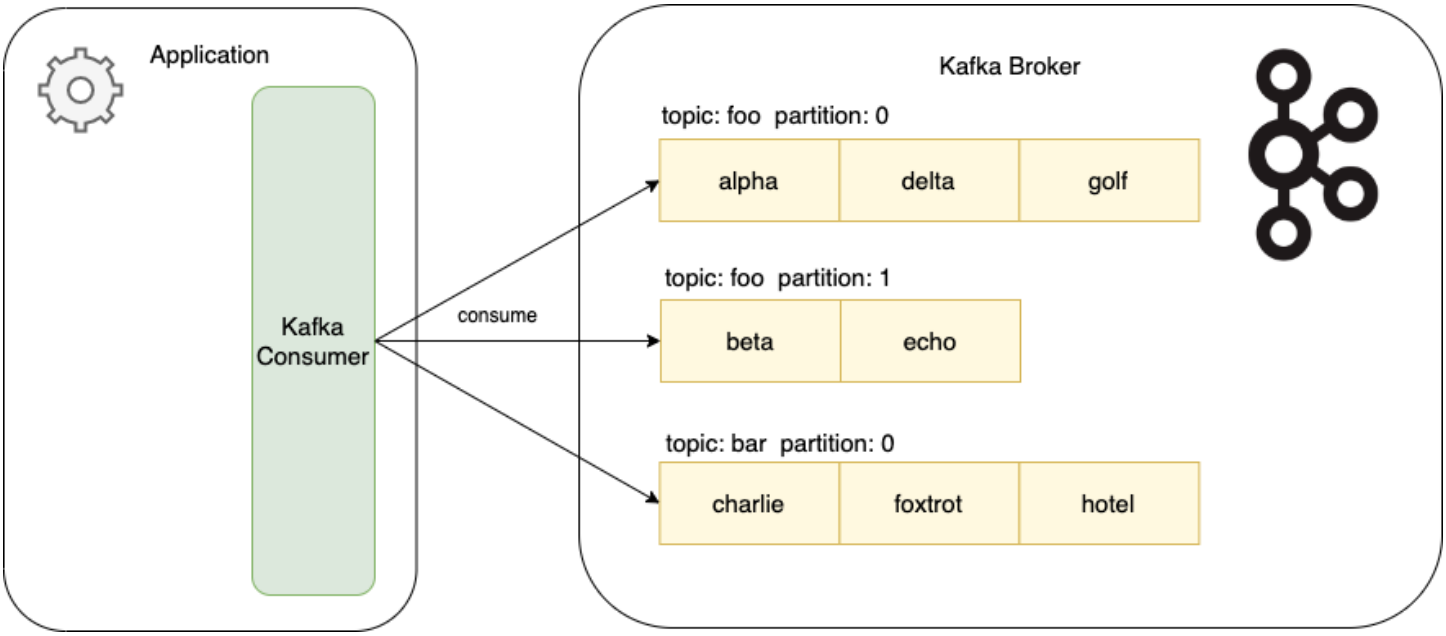

Figure 7: Consumer polling multiple topics and partitions

However the behaviour is the same as described in the three scenarios above, whether the happy path flow, the consumer fails, or one of the messages throws an exception. Essentially whether the messages come from the same partition or from multiple partitions it makes no difference to these flows. Once a message has completed processing then the consumer offsets are updated with the successfully consumed offset. In this example then three records would be written to the consumer offsets topic, marking each of the three partitions that have had a message consumed.

KafkaJS Behaviour

While this article and the companion project focus on the Java Apache Kafka client library behaviour, the same behaviour is observed with the Node.js Kafka library KafkaJS when using the **eachMessage** handler for processing. This again assumes that auto commit is enabled. Note that when using the **eachMessage** handler a batch of messages is still consumed by the library, with each then being passed one at a time to this handler by the KafkaJS library.

At the time of writing (August 2022) the latest version is v2.2.0. Within the consumer code in **runner.js** the function **processEachMessage(batch)** manages this behaviour. It is responsible for committing the offsets of any messages that have been successfully processed from within a batch when a subsequent message has thrown an exception.

https://github.com/tulios/kafkajs/blob/v2.2.0/src/consumer/runner.js#L250

```
// In case of errors, commit the previously consumed offsets unless autoCommit is disabled
await this.autoCommitOffsets()
```

Retries

In the scenario where the exception is thrown whilst processing a batch of messages, there would typically be two courses of action. If the exception is considered non-recoverable, then the message could be written to a dead letter topic for example. In our example the exception is being allowed to percolate up to the Kafka client library in order for the message to be retried.

There are different options to consider when configuring retry. These include stateless versus stateful retry, and factors such as exponential backoff and retry count. These are covered in the article Kafka Consumer Retry.

Spring Boot Demo

A companion Spring Boot application demonstrates the batch consume behaviour.

https://github.com/lydtechconsulting/kafka-batch-consume/tree/v1.0.0

The project includes a component test MessageBatchConsumeCT that spins up the dockerised Spring Boot application along with a dockerised Kafka, and is used to drive the message batch testing. The containers are managed by TestContainers with Lydtech's component-test-framework orchestrating the testing.

Conclusion

The Kafka client library ensures that whether an application fails during message processing, or an exception is thrown during processing mid-batch, that messages are not lost. Messages may however be redelivered resulting in duplicate processing. Each failure scenario needs to be understood so that the ramifications of duplicate processing can be mitigated for.